

'This' pointer and scopes

Those of us who have programmed in C++ are familiar with the *'this'* pointer, which is a special type that points to the object of which a non-static member function is being invoked. Let *'X'* be a class, and *my_A* an object of class *X*. We have a member function in *X* named *foo*.

```
class X
{
    int age;
    int birthyear;
    string my_name;
public:
    void set_birthyear(int year) {
        this->birthyear = year;
    }
    ....
    X my_A;
    // equivalent to my_A.set_birthyear(&my_A, 1971)
    my_A.set_birthyear(1971);
}
```

Consider the code snippet above. In the member function *set_birthyear*, we used *this->birthyear* to refer to the data member *birthyear* of the object on which the function *set_birthyear* is invoked. Note that we typically do not write *this->birthyear*, but just write the assignment as *birthyear = year*; since the usage of *'this'* is implicit. When you invoke *my_A.set_birthyear(1971)*, it is as if you have written the invocation as *my_A.set_birthyear(&my_A, 1971)*. The first argument is the *this* pointer, which is the pointer to the receiver object on which the function is invoked. Note that we cannot explicitly declare the *this* pointer, nor can we assign a different object to the *this* pointer in C++. It is not possible for me to pass a *my_B* object address as the *this* pointer in the above call, to *set_birthyear*. However, *'this'* behaves differently in JavaScript, and it would be useful to understand the difference. Before we discuss this, let us briefly explore scopes.

In C or C++, you can create a new scope by block statements. Whenever you open a new curly brace, it creates a new scope. For example, consider the following code:

```
int my_var = 1;
int main(void) {
    int my_var = 10;
    /* start a new scope here by a block */
    {
        int my_var = 20;
        int your_var = 30;
        //This prints 20 and 30
    }
}
```

```
printf("%d is my_var and %d is your_var \n",
my_var, your_var);
}
printf("%d is my_var", my_var); //This prints 10
/* this will give a compile time error saying undefined
variable
printf("%d is your_var", your_var); */;
}
```

In the above code snippet, there is a global named *my_var*. Inside the *main* function, we defined a function local called *my_var*, whose definition hides the global *my_var* definition. We then started a new *'block'* scope, and defined a block local variable *my_var*. The scope of the block-local variable is just the block where it is defined. Therefore, when we print the value of *my_var*, outside the block and inside *main*, we get the value of *my_var* as 10. The function-local *my_var* has the *main* function as its scope. On the other hand, in JavaScript, the only way of creating a scope is by means of functions. You can't use block statements represented by curly braces, or *for* loops, or anything else to create a new scope. The variables in JavaScript follow lexical scoping, which means that their scope is determined by the physical position of their definition in the source code—not by where the function is being called from.

Now let us get back to *'this'* in JavaScript. It refers to the current context, which is basically the receiver object of the method being invoked. Consider this JavaScript code snippet:

```
var Employee = {
    name: "Sandya",
    joinyear:2012,
    age:40,
    salutation: function () {
        return "Hi " + this.name + "You joined in" +
this.joinyear";
    }
};
Employee.salutation();
```

In the above code snippet, we defined a JavaScript object *Employee* and invoked the *salutation* function of that object. Inside the *salutation* function, we used the *'this'* context to refer to the receiver object (*Employee*, in this case). This is very similar to the behaviour you would see with C++. Now consider the following code snippet:

```
var bad_call = Employee.salutation();
bad_call();
```